

EECE7352 Computer Architecture

Homework 2

Sakthivel P. Sivakumar

NUID: 002312294

Part A1: a)

This part of the assignment deals with using the 32-bit base RISC V ISA (RV32I).

Condition 1: For a = 7, b = 8 and c = 9 (All inputs are different; output: largest among the 3)



RISC-V Simulator interface showing assembly code, registers, memory, and console output.

RISC-V Assembly:

```
0      addi    zero,zero,0
kernel:
1      addi    sp,zero,1536
2      call    main
3      addi    zero,zero,0
4      mv      s1,a0
5      addi    zero,zero,0
6      addi    zero,zero,0
7      auipc   ra,0x0
8      jalr    ra,0(ra)
9      addi    zero,zero,0
10     addi    zero,zero,0

largest:
11     addi    sp,sp,-48
12     sw      ra,44(sp)
13     sw      s0,40(sp)
14     addi    s0,sp,48
15     sw      a0,-36(s0)
16     sw      a1,-40(s0)
17     sw      a2,-44(s0)
18     lw      a4,-36(s0)
19     lw      a5,-40(s0)
20     bne     a4,a5,.L2
```

Registers:

Register	Value	Register	Value
zero [0]	0	ra [1]	12
sp [2]	1536	gp [3]	0
tp [4]	0	t0 [5]	0
t1 [6]	0	t2 [7]	0
s0/fp [8]	0	s1 [9]	9
a0 [10]	9	a1 [11]	8
a2 [12]	9	a3 [13]	0
a4 [14]	9	a5 [15]	9
a6 [16]	0	a7 [17]	0
s2 [18]	0	s3 [19]	0
s4 [20]	0	s5 [21]	0
s6 [22]	0	s7 [23]	0
s8 [24]	0	s9 [25]	0
s10 [26]	0	s11 [27]	0

Memory:

HEX	DEC	BINARY
STACK SEGMENT		
FREE SPACE		
0x000005fc	00 00 00 0c	
0x000005f8	00 00 00 00	
0x000005f4	00 00 00 00	
0x000005f0	00 00 00 00	
0x000005ec	00 00 00 07	
0x000005e8	00 00 00 08	
0x000005e4	00 00 00 09	
0x000005e0	00 00 00 00	
0x000005dc	00 00 01 ac	
0x000005d8	00 00 06 00	
0x000005d4	00 00 00 00	
0x000005d0	00 00 00 00	
0x000005cc	00 00 00 00	
0x000005c8	00 00 00 00	
0x000005c4	00 00 00 00	
0x000005c0	00 00 00 00	
0x000005bc	00 00 00 07	
0x000005b8	00 00 00 08	
0x000005b4	00 00 00 09	
0x000005b0	00 00 00 00	
0x000005ac	00 00 00 00	

Console:

```
***** Parser Output *****
Parsing successful!
```

Instruction breakdown:

31	12	11	7	6	0
imm[31:12]		rd		opcode	
000000000000000000000000		00001		0010111	

Condition 2: For a=7, b=9, c=9 (two equal valued largest inputs; output is 0)



RISC-V Simulator interface showing assembly code, registers, memory, and console output.

RISC-V Assembly:

```
0      addi    zero,zero,0
kernel:
1      addi    sp,zero,1536
2      call    main
3      addi    zero,zero,0
4      mv      s1,a0
5      addi    zero,zero,0
6      addi    zero,zero,0
7      auipc   ra,0x0
8      jalr    ra,0(ra)
9      addi    zero,zero,0
10     addi    zero,zero,0

largest:
11     addi    sp,sp,-48
12     sw      ra,44(sp)
13     sw      s0,40(sp)
14     addi    s0,sp,48
15     sw      a0,-36(s0)
16     sw      a1,-40(s0)
17     sw      a2,-44(s0)
18     lw      a4,-36(s0)
19     lw      a5,-40(s0)
20     bne     a4,a5,.L2
```

Registers:

Register	Value	Register	Value
zero [0]	0	ra [1]	12
sp [2]	1536	gp [3]	0
tp [4]	0	t0 [5]	0
t1 [6]	0	t2 [7]	0
s0/fp [8]	0	s1 [9]	0
a0 [10]	0	a1 [11]	9
a2 [12]	9	a3 [13]	0
a4 [14]	9	a5 [15]	0
a6 [16]	0	a7 [17]	0
s2 [18]	0	s3 [19]	0
s4 [20]	0	s5 [21]	0
s6 [22]	0	s7 [23]	0
s8 [24]	0	s9 [25]	0
s10 [26]	0	s11 [27]	0

Memory:

HEX	DEC	BINARY
STACK SEGMENT		
FREE SPACE		
0x000005fc	00 00 00 0c	
0x000005f8	00 00 00 00	
0x000005f4	00 00 00 00	
0x000005f0	00 00 00 00	
0x000005ec	00 00 00 07	
0x000005e8	00 00 00 09	
0x000005e4	00 00 00 09	
0x000005e0	00 00 00 00	
0x000005dc	00 00 01 ac	
0x000005d8	00 00 06 00	
0x000005d4	00 00 00 00	
0x000005d0	00 00 00 00	
0x000005cc	00 00 00 00	
0x000005c8	00 00 00 00	
0x000005c4	00 00 00 00	
0x000005c0	00 00 00 00	
0x000005bc	00 00 00 07	
0x000005b8	00 00 00 09	
0x000005b4	00 00 00 09	
0x000005b0	00 00 00 00	
0x000005ac	00 00 00 00	

Console:

```
***** Parser Output *****
Parsing successful!
```

Instruction breakdown:

31	12	11	7	6	0
imm[31:12]		rd		opcode	
000000000000000000000000		00001		0010111	

Condition 3: For a= 7, b=7, c=7 (All inputs are equal; output is square of any one input value)



RISC-V Simulator
Adaptive and Secure Computing Systems Lab • Boston University



ASCS | ADAPTIVE & SECURE
COMPUTING SYSTEMS
LABORATORY

RISC-V Assembly

```

0      addi    zero,zero,0
1      kernel:
2          addi    sp,zero,1536
3          call    main
4          addi    zero,zero,0
5          mv      s1,a0
6          addi    zero,zero,0
7          addi    zero,zero,0
8          auipc   ra,0x0
9          jalr    ra,0(ra)
10         addi    zero,zero,0
11         addi    zero,zero,0
12         largest:
13         addi    sp,sp,-48
14         sw      ra,44(sp)
15         sw      s0,40(sp)
16         addi    s0,sp,48
17         sw      a0,-36(s0)
18         sw      a1,-40(s0)
19         sw      a2,-44(s0)
20         lw      a4,-36(s0)
21         lw      a5,-40(s0)
22         bne     a4,a5,.L2
          
```

Console

```

***** Parser Output *****
Parsing successful!
          
```

Registers

Register	Value	Register	Value
zero [0]	0	ra [1]	12
sp [2]	1536	gp [3]	0
tp [4]	0	t0 [5]	0
t1 [6]	0	t2 [7]	0
s0/fp [8]	0	s1 [9]	49
a0 [10]	49	a1 [11]	7
a2 [12]	7	a3 [13]	0
a4 [14]	7	a5 [15]	49
a6 [16]	0	a7 [17]	0
s2 [18]	0	s3 [19]	0
s4 [20]	0	s5 [21]	0
s6 [22]	0	s7 [23]	0
s8 [24]	0	s9 [25]	0
s10 [26]	0	s11 [27]	0

Memory

HEX DEC BINARY

STACK SEGMENT

FREE SPACE

0x000005fc: 00 00 00 0c

0x000005f8: 00 00 00 00

0x000005f4: 00 00 00 00

0x000005f0: 00 00 00 00

0x000005ec: 00 00 00 07

0x000005e8: 00 00 00 07

0x000005e4: 00 00 00 07

0x000005e0: 00 00 00 00

0x000005dc: 00 00 01 ac

0x000005d8: 00 00 06 00

0x000005d4: 00 00 00 00

0x000005d0: 00 00 00 00

0x000005cc: 00 00 00 31

0x000005c8: 00 00 00 07

0x000005c4: 00 00 00 00

0x000005c0: 00 00 00 00

0x000005bc: 00 00 00 07

0x000005b8: 00 00 00 07

0x000005b4: 00 00 00 07

0x000005b0: 00 00 00 00

0x000005ac: 00 00 00 00

Instruction breakdown

31	12	11	7	6	0
imm[31:12]		rd		opcode	
00000000000000000000		00001		0010111	

b):

Among the three conditions discussed above, we get a large valued output (input^2) only when all the inputs are the same. We know that RV32I uses 32-bit signed integers ranged between -2^{31} to $(2^{31}) - 1$. We want to find the largest value that even when squared still fits in the signed 32-bit range.

Max Signed integer = $(2^{31}) - 1 = 2,147,483,647$

Sq. root of this value gives ~ 46,340.95 and the closest value that falls in the range without overflow is **46340**

2)

a) Octal to Binary



RISC-V Simulator
Adaptive and Secure Computing Systems Lab • Boston University



ASCS | ADAPTIVE & SECURE
COMPUTING SYSTEMS
LABORATORY

RISC-V Assembly

```

0      addi    zero,zero,0
1      kernel:
2          addi    sp,zero,1536
3          call    main
4          addi    zero,zero,0
5          mv      s1,a0
6          addi    zero,zero,0
7          addi    zero,zero,0
8          auipc   ra,0x0
9          jalr    ra,0(ra)
10         addi    zero,zero,0
11         addi    zero,zero,0
12         div8:
13         addi    sp,sp,-48
14         sw      ra,44(sp)
15         sw      s0,40(sp)
16         addi    s0,sp,48
17         sw      a0,-36(s0)
18         sw      zero,-20(s0)
19         j       .L3
          .L3:
20         lw      a5,-36(s0)
21         addi    a5,a5,-8
          
```

Console

```

***** Parser Output *****
Parsing successful!
          
```

Registers

Register	Value	Register	Value
zero [0]	0	ra [1]	12
sp [2]	1536	gp [3]	0
tp [4]	0	t0 [5]	0
t1 [6]	0	t2 [7]	0
s0/fp [8]	0	s1 [9]	1010
a0 [10]	1010	a1 [11]	1000000
a2 [12]	0	a3 [13]	0
a4 [14]	1000	a5 [15]	1010
a6 [16]	0	a7 [17]	0
s2 [18]	0	s3 [19]	0
s4 [20]	0	s5 [21]	0
s6 [22]	0	s7 [23]	0
s8 [24]	0	s9 [25]	0
s10 [26]	0	s11 [27]	0

Memory

HEX DEC BINARY

STACK SEGMENT

FREE SPACE

0x000005fc: 00 00 00 0c

0x000005f8: 00 00 00 00

0x000005f4: 00 00 00 00

0x000005f0: 00 00 00 00

0x000005ec: 00 00 00 0a

0x000005e8: 00 00 00 00

0x000005e4: 00 00 00 00

0x000005e0: 00 00 00 00

0x000005dc: 00 00 02 8c

0x000005d8: 00 00 06 00

0x000005d4: 00 00 00 00

0x000005d0: 00 00 00 00

0x000005cc: 00 00 00 00

0x000005c8: 00 00 00 00

0x000005c4: 00 00 00 01

0x000005c0: 00 00 00 00

0x000005bc: 00 00 00 02

0x000005b8: 00 00 00 01

0x000005b4: 00 00 00 0a

0x000005b0: 00 00 00 0a

0x000005ac: 00 00 03 e8

Instruction breakdown

31	12	11	7	6	0
imm[31:12]		rd		opcode	
00000000000000000000		00001		0010111	

b) Binary to Decimal

**risc-V Simulator**
Adaptive and Secure Computing Systems Lab • Boston University

**ASCS** | ADAPTIVE & SECURE
COMPUTING SYSTEMS
LABORATORY

RISC-V Assembly

110

jr

ra

main:

111

addi

sp,sp,-32

112

sw

ra,28(sp)

113

sw

s0,24(sp)

114

addi

s0,sp,32

115

li

a5,12

116

sw

a5,-20(s0)

117

li

a1,1

118

lw

a0,-20(s0)

119

mv

s2,a0#input

120

call

oct2dec

121

mv

s3,a0#store output

122

mv

a5,a0

123

mv

a0,a5

124

lw

ra,28(sp)

125

lw

s0,24(sp)

126

addi

sp,sp,32

127

jr

ra

128

addi

zero,zero,0

129

addi

zero,zero,0

130

addi

zero,zero,0

131

addi

zero,zero,0

Registers

Register	Value	Register	Value
zero [0]	0	ra [1]	12
sp [2]	1536	gp [3]	0
tp [4]	0	t0 [5]	0
t1 [6]	0	t2 [7]	0
s0/fp [8]	0	s1 [9]	12
a0 [10]	12	a1 [11]	64
a2 [12]	0	a3 [13]	0
a4 [14]	8	a5 [15]	12
a6 [16]	0	a7 [17]	0
s2 [18]	12	s3 [19]	12
s4 [20]	0	s5 [21]	0
s6 [22]	0	s7 [23]	0
s8 [24]	0	s9 [25]	0
s10 [26]	0	s11 [27]	0

Memory

HEX

DEC

BINARY

HEX	DEC	BINARY
STACK SEGMENT		
FREE SPACE		
0x000005fc	00 00 00 0c	
0x000005f8	00 00 00 00	
0x000005f4	00 00 00 00	
0x000005f0	00 00 00 00	
0x000005ec	00 00 00 0c	
0x000005e8	00 00 00 00	
0x000005e4	00 00 00 00	
0x000005e0	00 00 00 00	
0x000005dc	00 00 01 e4	
0x000005d8	00 00 06 00	
0x000005d4	00 00 00 00	
0x000005d0	00 00 00 00	
0x000005cc	00 00 00 04	
0x000005c8	00 00 00 01	
0x000005c4	00 00 00 04	
0x000005c0	00 00 00 08	
0x000005bc	00 00 00 0c	
0x000005b8	00 00 00 01	
0x000005b4	00 00 00 00	
0x000005b0	00 00 00 00	
0x000005ac	00 00 01 9c	

Console

***** Parser Output *****

Parsing successful!

Instruction breakdown

31	12	11	7	6	0
imm[31:12]		rd		opcode	
00000000000000000000		00001		0010111	

Thus, both the implementation followed the RISC-V calling conventions, where arguments are passed in registers (a0 – a2) and the results are returned in a0. Since multiplication is not available in RV32I, repeated addition was used to compute square.

Similarly in the 2nd problem where I was required to convert numbers between different bases using recursion. To avoid multiplication, division, and modulus, helper routines were implemented using subtraction and repeated addition. This ensured that all generated instructions remained within the RV32I base instruction set.